Weapon-Target Assignment Problem

CSE 462: Algorithm Engineering Sessional

Final Presentation

Team Members

Hozifa Rahman Hamim	(ID: 1705083)
Md. Azizul Haque Nadim	(ID: 1905059)
Sadia Tabassum	(ID: 1905091)
Ahmmad Nur Swapnil	(ID: 2005009)
Most. Sonia Khatun	(ID: 2005025)
Shahriar Ahmed Seam	(ID: 2005093)

Department of Computer Science and Engineering (CSE)

Bangladesh University of Engineering and Technology (BUET)

30th March 2026

Section 1

Problem Definition

Formal definition & mathematical formulation

Complexity class & hardness analysis

Real-World Motivation: Iron Dome

Israel's Missile Defense System

Iron Dome uses **WTA algorithms** to assign interceptor missiles to incoming rockets in real time, achieving a ≈**90% interception rate** under extreme time pressure.

Problem Mapping

Incoming Rockets $\longrightarrow$ **Targets** (T)

Interceptor Missiles $\longrightarrow$ **Weapons** (W)

Optimal Pairing $\longrightarrow$ **WTA Solution** (X)

Why WTA Matters

- **Real-time decisions** — seconds to react
- **Scarce resources** — fewer interceptors than threats
- **Priority targets** — protect high-value assets first

Core Question:

Which weapon engages which target to minimize total expected damage?

Presentation Outline

Part 1: Foundations

1. **Problem Definition**
   - Formal definition & mathematical formulation
   - Complexity class & hardness analysis
2. **Algorithm Survey**
   - Overview of existing approaches
   - Academic references & brief descriptions
3. **Chosen Algorithms in Detail**
   - Step-by-step explanation of MMR algorithm
   - Step-by-step explanation of Genetic algorithm
   - Visual illustrations for clarity

Part 2: Contributions

4. **Proposed Modifications**
   - Improvements to selected algorithms
   - Reasoning & theoretical insights
5. **Experimental Setup**
   - Dataset design and scenario construction
   - Benchmark protocol and comparison settings
6. **Experimental Results**
   - Original vs. modified algorithm outcomes
   - Runtime-quality tradeoff and key findings

Problem Definition

Weapon-Target Assignment (WTA) Problem

Optimally allocate weapons to targets to **minimize residual threat** under real-time operational constraints.

Core Challenge: Stochastic Combat

Weapon w_i destroys target t_j with kill probability $p_{ij} \in [0, 1]$. Assignments must be optimized **under uncertainty**, in real time.

Computational Complexity

- Proven **NP-complete** — no polynomial-time exact solution exists
- Search space grows **exponentially** with weapons and targets
- Large instances require **heuristic or metaheuristic** approaches

Classification: Asset-based vs. Target-based

Two Perspectives of WTA

WTA is framed by two objectives: **destroy enemy forces** or **protect friendly assets**.

Target-based

Values assigned to each **enemy target** based on threat they pose.

Goal: Maximize total damage inflicted on incoming targets.

Asset-based

Values assigned to **defended assets** (cities, bases) rather than targets.

Goal: Maximize combined survival value of friendly assets.

Mathematical Representation: Parameters and Variables

Target-based Formulation – Parameters

- $W = \{1, \ldots, m\}$: The set of available **weapon types**
- $T = \{1, \ldots, n\}$: The set of **targets** to be engaged
- v_j: The **strategic military value** or importance of target j
- p_{ij}: The **probability of kill** — chance that one weapon of type i will destroy target j
- w_i: The **total quantity** of weapons of type i available

Decision Variable

$$x_{ij} \in \{0, 1, 2, \ldots\}$$

Integer variable representing the number of weapons of type i assigned to target j

Mathematical Representation: Objective Functions

Goal: Maximize damage (or equivalently, minimize survival)

While the goal is to maximize damage, the standard convention is to minimize remaining survival value.

Maximizing Total Expected Damage

$$\text{Max } Z = \sum_{j=1}^{n} v_j \left[1 - \prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}} \right]$$

Minimizing Expected Survival Value

$$\text{Min } Z = \sum_{j=1}^{n} v_j \prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}}$$

Key Insight

$\prod_{i=1}^{m} (1 - p_{ij})^{x_{ij}}$ = **Probability that target j survives ALL assigned weapons**

The product captures the **combined effect** of multiple weapons attacking the same target.

Mathematical Representation: Constraints

Constraint 1: Resource Availability

$$\sum_{j=1}^{n} x_{ij} \leq w_i, \quad \forall i \in W$$

Meaning: Total weapons of type i assigned across all targets cannot exceed available quantity w_i

Constraint 2: Integrality

$$x_{ij} \in \{0, 1, 2, \ldots\}$$

Meaning: We can only assign **whole numbers** of weapons (discrete assignments)

Complete Formulation Summary

Minimize $\sum_j v_j \prod_i (1 - p_{ij})^{x_{ij}}$ **s.t.** $\sum_j x_{ij} \leq w_i, \quad x_{ij} \in \mathbb{Z}^+$

WTA Decision Problem

? Decision Version of WTA

Given:

- Survival probability $q_{i,j} = 1 - p_{i,j}$
- Threshold value k

Question: Does there exist an assignment $x_{i,j} \in \{0, 1\}$ such that:

$$D = \sum_{j=1}^{n} V_j \prod_{i=1}^{m} q_{i,j}^{x_{i,j}} \leq k$$

Subject to Constraints

$$\sum_{j=1}^{n} x_{i,j} \leq W_i \quad \text{for } i = 1, \ldots, m \qquad x_{i,j} \in \{0, 1\}$$

where $x_{i,j} = 1$ if weapon i assigned to target j, and $x_{i,j} = 0$ otherwise.

D represents **total expected damage** (weighted survival probability of all targets).

Two Polynomial-Time Reductions

Our Approach

We demonstrate two polynomial-time reductions:

Reduction 1

SUBSET PRODUCT $\leq_p$ WTA

$\Rightarrow$ Proves **WTA is NP-hard**

Reduce FROM a known NP-complete problem TO WTA.

Reduction 2

WTA $\leq_p$ Multiple Knapsack

$\Rightarrow$ Shows WTA reduces to another NP-hard problem

Shows connection between problems.

Reduction 1: SUBSET PRODUCT $\leq_p$ WTA

SUBSET PRODUCT Problem (NP-complete)

Given: Set $A = \{a_1, a_2, \ldots, a_m\}$ and target product b
Find: Subset $X \subseteq A$ where $\prod_{x \in X} a_x = b$

WTA Instance Construction

Let $P = \prod_{i=1}^{m} a_i$. Construct WTA with:

- **Targets:** 2 targets with values $V_1 = V_2 = P$
- **Weapons:** $m + 2$ weapons: $W_1, W_2, \ldots, W_m, W_\alpha, W_\beta$
- **Survival probabilities:** $q_{i,1} = q_{i,2} = \frac{1}{a_i}$ for $i = 1, \ldots, m$; $\quad q_{\alpha,1} = q_{\alpha,2} = \frac{b}{2P}$; $\quad q_{\beta,1} = q_{\beta,2} = \frac{1}{2b}$
- **Threshold:** $k = 1$ (we ask if $D \leq 1$)

Time Complexity

Constructs WTA with 2 targets and $m + 2$ weapons in $O(m)$ time — **Polynomial!**

Reduction 1: Proof of Correctness

Forward Direction ($\Rightarrow$): SUBSET PRODUCT has solution $\Rightarrow$ WTA has solution

Suppose $X \subseteq \{1, \ldots, m\}$ with $\prod_{x \in X} a_x = b$. Let $Y = \{1, \ldots, m\} \setminus X$. Then:

$$\prod_{i \in X} \frac{1}{a_i} = \frac{1}{b}, \qquad \prod_{y \in Y} \frac{1}{a_y} = \frac{b}{P}$$

Assignment: $\{W_x : x \in X\} \cup \{W_\alpha\} \to T_1$ and $\{W_y : y \in Y\} \cup \{W_\beta\} \to T_2$
Total damage:

$$D = P \cdot \frac{b}{2P} \cdot \frac{1}{b} + P \cdot \frac{1}{2b} \cdot \frac{b}{P} = \frac{1}{2} + \frac{1}{2} = 1 \leq k \quad \checkmark$$

Backward Direction ($\Leftarrow$): WTA has solution $\Rightarrow$ SUBSET PRODUCT has solution

Suppose WTA has solution with $D \leq 1$. Let X', Y' be weapons assigned to T_1, T_2.
Claim: W_α and W_β cannot both target the same target.
Proof: If both assigned to T_1 (symmetric for T_2):

$$D \geq P \cdot \frac{1}{2b} \cdot \frac{b}{2P} \cdot \frac{1}{\prod_{x \in X' \setminus \{\alpha, \beta\}} a_x} + \frac{P}{\prod_{y \in Y'} a_y} \geq \frac{1}{4} + 1 > 1 \implies \Leftarrow$$

Reduction 1: Backward Direction (Continued)

From the Assignment

With $W_\alpha \to T_1$ and $W_\beta \to T_2$:

$$D = P \cdot \frac{b}{2P} \prod_{x \in X} \frac{1}{a_x} + P \cdot \frac{1}{2b} \prod_{y \in Y} \frac{1}{a_y} \leq 1$$

Algebraic Derivation

Let $z = \prod_{x \in X} a_x$. Then:

$$\frac{b}{2z} + \frac{z}{2b} \leq 1$$

Multiply by $2bz > 0$:

$$b^2 + z^2 \leq 2bz \implies (b - z)^2 \leq 0$$

Since $(b - z)^2 \geq 0$, we must have $(b - z)^2 = 0$, so:

$$b = z = \prod_{x \in X} a_x$$

Conclusion

X is a valid solution to SUBSET PRODUCT! ✓ **Time Complexity:** $O(m)$ — Polynomial reduction complete.

Reduction 2: WTA $\leq_p$ Multiple Knapsack

Multiple Knapsack Construction

Given WTA with m targets, n weapons, probabilities $\{p_{i,j}\}$:

- **Knapsacks:** m knapsacks (one per target)
- **Capacity:** $C_j = \infty$ (or sufficiently large)
- **Items:** n items (one per weapon type)
- **Profit:** $v_{ij} = -\log(1 - p_{i,j}) = -\log(q_{i,j})$
- **Weight:** $w_{ij} = 0$ (or 1 if weapon limits exist)

Objective Transformation (Key Insight)

For single target j, minimizing $\prod q_{i,j}^{x_{i,j}}$ equals minimizing:

$$\log\left(\prod_{i=1}^{n} q_{i,j}^{x_{i,j}}\right) = \sum_{i=1}^{n} x_{i,j} \log(q_{i,j})$$

Equivalent to **maximizing**:

$$-\sum_{i=1}^{n} x_{i,j} \log(q_{i,j}) = \sum_{i=1}^{n} x_{i,j} \cdot v_{ij}$$

This is **exactly the Multiple Knapsack objective!**

Reduction 2: Constraint Mapping & Conclusion

Constraint Mapping

- **WTA constraint:** $\sum_{j=1}^{m} x_{i,j} \leq W_i$ (weapon type i used at most W_i times)
- **Maps to:** $\sum_{j=1}^{m} y_{ij} \leq 1$ (each item in at most one knapsack) when $W_i = 1$
- For general W_i: Create W_i copies of item i

Equivalence

Assignment $x_{i,j}$ optimal for WTA $\Leftrightarrow$ $y_{ij} = x_{i,j}$ optimal for Multiple Knapsack
Logarithmic transformation preserves ordering of solutions!

Time: $O(nm)$ to compute all $v_{ij} = -\log(1 - p_{i,j})$

Final Conclusion

WTA is in NP (solution verifiable in polynomial time) + **WTA is NP-hard**

$\Rightarrow$ **The Weapon-Target Assignment Problem is NP-complete!**

Section 2

Existing Algorithms Survey

Five Major Categories of Solution Approaches

Existing Algorithms Survey

Comprehensive Algorithm Categories for WTA

Five major categories of solution approaches

- **Exact:** Optimal solutions, but exponential time complexity.
- **Approximation:** Provable bounds within polynomial time.
- **Randomized:** Probabilistic search where quality $\propto$ samples.
- **Heuristic:** Fast, problem-specific, and near-optimal.
- **Metaheuristic:** General-purpose optimization with exploration balance.

Category 1: Exact Exponential Algorithms

Guaranteed Optimal Solutions

Algorithm	Description	Complexity	Ref
Branch-and-Bound	LP/IP relaxations with MMR branching logic	$O(2^{mn})$	[1]
Branch-Price-Cut	Column generation for large-scale variables	$O(nm \cdot K)$	[2]
Branch-and-Adjust	Compact linear under-approximation	Exp. Worst	[3]

Category 2: Approximation Algorithms

Provable Bounds, Polynomial Time

Algorithm	Description	Complexity	Ref
Piecewise Linear	Nonlinear survival probability approximation	$O(mnB)$	[4,3]
LP Relaxation	Continuous relaxation followed by rounding	$O(poly)$	[1]
Lagrangian	Independent target subproblem decomposition	$O(Tnm)$	[5,6]

Category 3: Randomized Algorithms

Probabilistic Exploration

Algorithm	Description	Complexity	Ref
Monte Carlo	Random sampling; evaluates survival rates	$O(Smn)$	[7]
Randomized Heuristic	Probabilistic selection via Marginal Return	$O(Rmn)$	[8]

Category 4: Heuristic Algorithms

Fast, Near-Optimal Search

Algorithm	Description	Complexity	Ref
MMR	Greedy; sequential assignment based on return	$O(mn)$	[1,9]
VLSN	Improvement graphs; multi-exchange moves	$O(m^2n^2)$	[1]
MRBCH	Weapon-target-sensor triplet optimization	$O(mn \log mn)$	[10]

Category 5: Metaheuristic Algorithms

General-Purpose Optimization

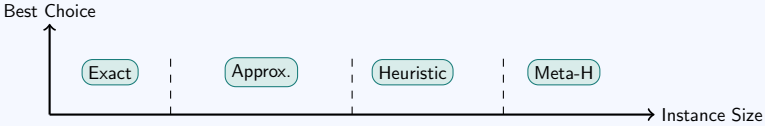
Algorithm	Description	Complexity	Ref
Genetic (GA)	Crossover and mutation population search	$O(GPmn)$	[11,12]
Ant Colony (ACO)	Pheromone construction and local search	$O(IAmn)$	[13,14]
Modified Pareto	Multi-objective optimization for WTA	$O(IAmnK)$	[15]

Complexity Comparison Summary

When to Use Each Category?

Choice depends on instance size, time constraints, and quality requirements.

Category	Complexity	Instance Limit	Use Case
Exact	$O(2^{mn})$	< 1000 variables	Global Optimality
Approximation	$O(poly(m, n))$	> 10000	Theoretical Bounds
Randomized	$O(S \cdot mn)$	Any (quality $\propto S$)	Baselines
Heuristic	$O(mn) - O(m^2 n^2)$	> 1000	Real-time Systems
Metaheuristic	$O(I \cdot P \cdot mn)$	> 10000	Large-scale Optimization



Section 3

MMR & Genetic Algorithm

Step-by-step illustration with a 3×3 instance

Problem Instance

Target Values

	Target 0	Target 1	Target 2
Value	10	20	30

Kill Probabilities p[weapon][target]

	Target 0	Target 1	Target 2
Weapon 0	0.3	0.2	0.1
Weapon 1	0.1	0.4	0.2
Weapon 2	0.2	0.3	**0.5**

Objective

Minimize total survival value of all targets

$$\sum_j V_j \cdot \prod_i (1 - p_{ij})$$

Each weapon assigned to exactly one target

Both algorithms solve the same problem — they differ in how they search for the optimal allocation.

MMR Algorithm — Overview

1. **Scan all pairs**
   For every unallocated weapon w and every target t:
   $\text{decrease} = V[t] \times p[w][t]$
2. **Pick best pair**
   Select weapon-target with *maximum* decrease "most bang for the buck"
3. **Assign & update**
   Assign weapon → target. $V[t] -= \text{maxDecrease}$
   Remove weapon from pool.
4. **Repeat**
   Return to step 1 until all weapons assigned.

Key Properties

Complexity: $O(W^2 \times T)$

Deterministic: same input → same output

Optimality: local optimum only

Speed: very fast, single pass

Greedy commits to the locally best choice each round — fast but may miss the globally optimal solution.

MMR Algorithm — Pseudocode

Algorithm 1 MMR Algorithm

```
solution.Allocations ← {}
solution.Value ← MaxValue
allocatedWeaponCount ← 0
while allocatedWeaponCount < numOfWeapons do
    maxDecrease ← 0;   k ← 1
    while k ≤ numOfTargets do
        decrease ← solution.Value[k] × KillProbabilities[unallocatedWeapon][k]
        if decrease > maxDecrease then
            maxDecrease ← decrease
            allocatedTarget ← k
        end if
        k ← k + 1
    end while
    unallocatedWeapons.Remove(allocatedWeapon)
    solution.Allocation[allocatedWeapon] ← allocatedTarget
    solution.Value[allocatedTarget] −= maxDecrease
    allocatedWeaponCount ← allocatedWeaponCount + 1
end while
solution.value ← CalculateSolutionValue(solution, allocations)
return solution
```

MMR — Round 1: Find Maximum Decrease

Scan all pairs. decrease = current_value[t] $\times$ kill_prob[w][t]

Weapon	Target	Calculation	Decrease
W0	T0	10×0.3	3.0
W0	T1	20×0.2	4.0
W0	T2	30×0.1	3.0
W1	T0	10×0.1	1.0
W1	T1	20×0.4	8.0
W1	T2	30×0.2	6.0
W2	T0	10×0.2	2.0
W2	T1	20×0.3	6.0
W2	**T2**	$\mathbf{30 \times 0.5}$	**15.0 ← MAX**

Result

W2 → T2

Decrease: 15.0

Update:

$V[\text{T2}] = 30 - 15 = 15$

W2 removed from pool

Remaining: W0, W1

MMR — Rounds 2 & 3 → Final Result

Round 2 — W: W0,W1 — V: T0=10,T1=20,T2=1

W×T	Calc	Dec
W0→T0	10×0.3	3.0
W0→T1	20×0.2	4.0
W0→T2	15×0.1	1.5
W1→T0	10×0.1	1.0
W1→T1	$\mathbf{20 \times 0.4}$	**8.0←**
W1→T2	15×0.2	3.0

W1 → T1 $V[\text{T1}] = 20 - 8 = 12$

Round 3 — W: W0 — V: T0=10,T1=12,T2=15

W×T	Calc	Dec
W0→T0	$\mathbf{10 \times 0.3}$	**3.0←**
W0→T1	12×0.2	2.4
W0→T2	15×0.1	1.5

W0 → T0

Final MMR Allocation

W0→T0 (p=0.3) W1→T1 (p=0.4) W2→T2 (p=0.5)

allocation = [0, 1, 2]

Genetic Algorithm — Overview

Chromosome Encoding

`chromosome[i] = target assigned to weapon` i

$[2, 0, 1] \Rightarrow W0 \rightarrow T2,\ \ W1 \rightarrow T0,\ \ W2 \rightarrow T1$

Population

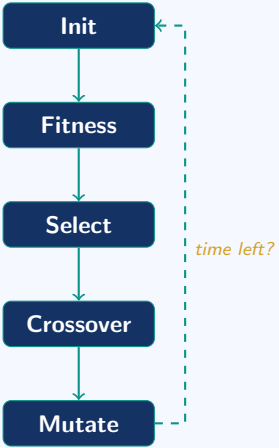
$\texttt{pop_size} = \max(W,\ T) = 3$

3 random chromosomes initially

Parameters

Parameter	Value
$P_{\text{crossover}}$	0.8
Tournament k	3
P_{mutation}	$1/W = 0.33$
pop_size	$\max(W,\ T) = 3$

Evolutionary Loop



Loop repeats until time budget exhausted.

Best solution is tracked across all generations.

Genetic Algorithm — Pseudocode

Algorithm 2 Genetic Algorithm

```
startTime ← NOW
endTime ← startTime + allowedSearchTime
solution.Allocations ← {}
solution.value ← MAXVALUE
if noOfTargets > noOfWeapons then
    noOfIndividuals ← noOfTargets
else
    noOfIndividuals ← noOfWeapons
end if
population ← GENERATEINITIALPOPULATION(noOfIndividuals)
individualNo ← 1
while CURRENTTIME < endTime do
    indFromIndiv ← population[individualNo]
    indValue ← CALCULATESOLUTIONVALUE(indFromIndiv)
    if indValue < solution.value then
        solution.value ← indValue;   solution ← indFromIndiv
    end if
    individualNo ← individualNo + 1
    if individualNo > noOfIndividuals then
        parents ← SELECTFROMINDIV(population)
        offspring ← CROSSOVER(parents)
        offspring ← MUTATE(offspring)
        population ← offspring;   individualNo ← 1
    end if
end while
return solution
```

GA Realistic Simulation: Problem Setup

Problem Parameters

Problem Size:

- **Weapons:** $W = 4$ (W0, W1, W2, W3)
- **Targets:** $T = 3$ (T0, T1, T2)

Target Values:

- $V_0 = 15$
- $V_1 = 25$
- $V_2 = 40$

Kill Probability Matrix p_{ij}:

	T0	**T1**	**T2**
W0	0.4	0.3	0.2
W1	0.2	0.5	0.3
W2	0.3	0.4	0.6
W3	0.5	0.2	0.4

GA Parameters:

- Population size $= 4$
- Tournament k $= 3$
- Crossover rate $= 0.8$
- Mutation rate $= 1/4 = 0.25$

Step 1: Initialize Population & Evaluate Fitness

Random Initial Population (4 individuals)

Individual 0: [1, 2, 0, 1]

W0→T1, W1→T2, W2→T0, W3→T1

Individual 1: [0, 1, 2, 0]

W0→T0, W1→T1, W2→T2, W3→T0

Individual 2: [2, 0, 1, 2]

W0→T2, W1→T0, W2→T1, W3→T2

Individual 3: [1, 1, 2, 0]

W0→T1, W1→T1, W2→T2, W3→T0

Fitness Calculation Example (Ind 1):

Chromosome: [0, 1, 2, 0]

$$\text{T0: } 15 \times (1 - 0.4) \times (1 - 0.5) = 15 \times 0.3 = 4.5$$
$$\text{T1: } 25 \times (1 - 0.5) = 12.5$$
$$\text{T2: } 40 \times (1 - 0.6) = 16.0$$

$$f_1 = 4.5 + 12.5 + 16.0 = \mathbf{33.0}$$

All Fitnesses:

Ind	Chrom	Fitness
0	[1,2,0,1]	45.2
1	**[0,1,2,0]**	**33.0 ***
2	[2,0,1,2]	38.8
3	[1,1,2,0]	36.5

Step 2: Tournament Selection (k=3)

Select 2 Parents using Tournament

Tournament 1 (Select Parent 1):

Randomly pick 3 individuals: **0, 2, 3**

Ind	Chrom	Fitness
0	[1,2,0,1]	45.2
2	[2,0,1,2]	38.8
3 *	**[1,1,2,0]**	**36.5**

P1 = [1, 1, 2, 0] (Ind 3)

Tournament 2 (Select Parent 2):

Randomly pick 3 individuals: **1, 2, 3**

Ind	Chrom	Fitness
1 *	**[0,1,2,0]**	**33.0**
2	[2,0,1,2]	38.8
3	[1,1,2,0]	36.5

P2 = [0, 1, 2, 0] (Ind 1)

Result: P1 = [1,1,2,0], P2 = [0,1,2,0] (Different!)

Step 3: Uniform Crossover (p_crossover = 0.8)

Apply Crossover

Random number: $0.65 < 0.8 \rightarrow$ **Apply crossover** ✓

Uniform Crossover Mechanism:

Each gene independently chooses from P1 or P2 based on random mask

Gene	W0	W1	W2	W3
P1	1	1	2	0
P2	0	1	2	0
Random mask (T=P1, F=P2)	T	F	T	F
C1	**1**	**1**	**2**	**0**
C2	**0**	**1**	**2**	**0**

Child 1 = [1, 1, 2, 0]

Child 2 = [0, 1, 2, 0]

Step 4a: Mutation on Child 1 (p_mutation = 0.25)

Mutate Child 1 = [1, 1, 2, 0]

Each gene mutates with probability 0.25(if equal or less than mutate)

Gene	W0	W1	W2	W3
Before	1	1	2	0
Random	0.82	0.15	0.67	0.91
¡ 0.25?	No	**Yes**	No	No
Action	keep	**mutate!**	keep	keep
New target	-	**0**	-	-
After	**1**	**0**	**2**	**0**

Mutated C1 = [1, 0, 2, 0]

Gene W1 mutated: $1 \rightarrow 0$ (changed from T1 to T0)

Step 4b: Mutation on Child 2 (p_mutation = 0.25)

Mutate Child 2 = [0, 1, 2, 0]

Each gene mutates with probability 0.25(if equal or less than mutate)

Gene	W0	W1	W2	W3
Before	0	1	2	0
Random	0.44	0.73	0.19	0.28
¡ 0.25?	No	No	**Yes**	No
Action	keep	keep	**mutate!**	keep
New target	-	-	**1**	-
After	**0**	**1**	**1**	**0**

Mutated C2 = [0, 1, 1, 0]

Gene W2 mutated: $2 \rightarrow 1$ (changed from T2 to T1)

Step 5: Evaluate Offspring Fitness

Calculate Fitness of Both Children

Child 1: [1, 0, 2, 0]

W0→T1, W1→T0, W2→T2, W3→T0

$$\begin{aligned} \text{T0: } & 15 \times (1-0.2)(1-0.5) \\ & = 15 \times 0.4 = 6.0 \\ \text{T1: } & 25 \times (1-0.3) = 17.5 \\ \text{T2: } & 40 \times (1-0.6) = 16.0 \end{aligned}$$

$$f_{C1} = 6.0 + 17.5 + 16.0 = 39.5$$

Child 2: [0, 1, 1, 0]

W0→T0, W1→T1, W2→T1, W3→T0

$$\begin{aligned} \text{T0: } & 15 \times (1-0.4)(1-0.5) \\ & = 15 \times 0.3 = 4.5 \\ \text{T1: } & 25 \times (1-0.5)(1-0.4) \\ & = 25 \times 0.3 = 7.5 \\ \text{T2: } & 40 \times 1.0 = 40.0 \end{aligned}$$

$$f_{C2} = 4.5 + 7.5 + 40.0 = 52.0$$

Comparison:

Individual	Chromosome	Fitness
Child 1	[1,0,2,0]	**39.5 ***
Child 2	[0,1,1,0]	52.0

Child 1 is better! (Lower fitness = less survival = better solution)

Step 6: Update Best Solution

Compare with Current Best

Before this generation:

Item	Value
Best chromosome	[0,1,2,0]
Best fitness	**33.0**

(This was Individual 1 from initial population)

After this generation:

Item	Value
Best chromosome	[0,1,2,0]
Best fitness	**33.0**

Child 1 (39.5) and Child 2 (52.0) are both worse than 33.0

No improvement this generation

Result:

Best solution remains: [0, 1, 2, 0] with fitness 33.0

This is normal in GA - not every generation improves. The evolutionary process continues...

Step 7: Replace Population & Continue

New Population for Next Generation

Standard GA: Replace old population with offspring

Old Population (Generation 0):

Ind	Chrom	Fit
0	[1,2,0,1]	45.2
1	[0,1,2,0]	33.0 *
2	[2,0,1,2]	38.8
3	[1,1,2,0]	36.5

New Population (Generation 1):

Ind	Chrom	Fit
0	[1,0,2,0]	39.5
1	[0,1,1,0]	52.0
2	...	...
3	...	...

(2 more offspring created similarly)

Important Observation:

The best solution (33.0) from Generation 0 was LOST! This is why **elitism** is important in Hybrid GA.

The evolutionary loop continues until time budget expires...

MMR vs GA — Side-by-Side Comparison

	MMR	Genetic Algorithm
Approach	Greedy — one best step at a time	Evolutionary — population-based search
Speed	Very fast $O(W^2 \times T)$	Slower — time-budget driven
Optimality	Local optimum only	Closer to global optimum
Fitness	(compute via formula)	(compute via formula)
Randomness	Deterministic	Stochastic (seed-based)

GA finds better solution by exploring many chromosomes. MMR is faster but commits to early greedy choices.

Section 4

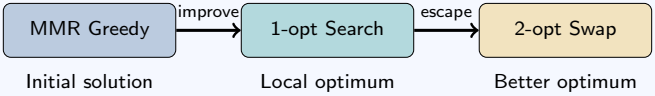
Our Modifications

Improving MMR and GA with Novel Enhancements

Modification 1: MMR-IR (Iterative Refinement)

Overview: Three Enhancements to MMR

The greedy MMR solution is a good starting point but may be suboptimal because early decisions cannot be reversed. Our modifications add **post-processing local search** to improve the greedy solution.



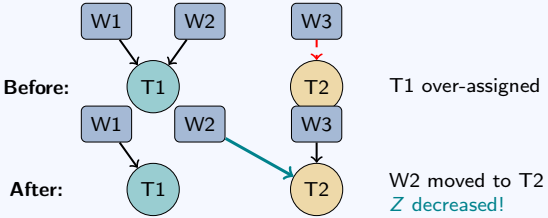
Enhancement 1: Tie-Breaking by Target Value

When two weapon-target pairs have **equal marginal return**, prefer the pair targeting the **higher-value target**. This prioritises coverage of critical targets.

MMR-IR: 1-opt Local Search

Enhancement 2: 1-opt Incremental Local Search

After MMR greedy completes, try **reassigning each weapon to every other target**. Accept if the objective improves. Repeat until no improvement is found.



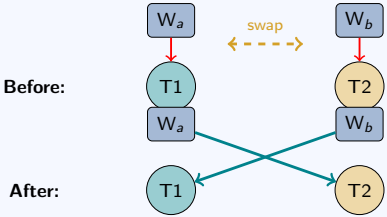
Incremental Evaluation

Uses $O(1)$ delta computation per trial (survival array maintained via multiplication/division) instead of $O(WT)$ full recomputation. Max 20 passes until convergence.

MMR-IR: 2-opt Swap Search

Enhancement 3: 2-opt Swap Search

After 1-opt converges, try **swapping the targets of two weapons simultaneously**. This can escape 1-opt local optima that single reassignments cannot.



Why 2-opt Helps

- 1-opt is stuck: moving W_a alone worsens T1; moving W_b alone worsens T2
- Swapping **both simultaneously** can improve the total objective
- Max 3 passes to limit runtime; incremental $O(1)$ delta per trial

Modification 2: Hybrid GA — Overview

Three Weaknesses of Standard GA on WTA

1. **Random initialization** — in high-dimensional space ($W \times T$ variables), random solutions are far from optimal
2. **No elitism** — the best solution found can be **lost** when replaced by offspring
3. **Blind mutation** — random target reassignment ignores which targets **need** more weapons

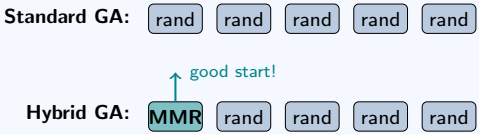
Our Three Fixes

Weakness	**Our Fix**	**Type**
Random initialisation	MMR-seeded init	Warm start
No elitism / stagnation	Elitism + detection	Preservation
Blind mutation	Threat-proportional	**WTA-specific**

Hybrid GA Mod 1: MMR-Seeded Initialisation

Idea: Start from a Good Solution

Instead of initialising **all** individuals randomly, seed `population[0]` with the MMR greedy solution. The rest remain random for diversity.



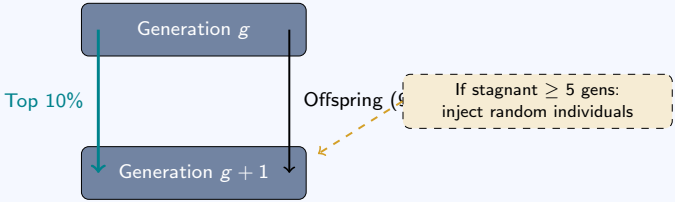
Why This Works

- MMR gives a **near-optimal** starting point in $O(W^2 T)$ time
- GA then **refines** this via crossover and mutation
- Random individuals provide **genetic diversity** to avoid local optima
- For large instances ($W = 375$), this saves hundreds of wasted generations

Hybrid GA Mod 2: Elitism + Stagnation Detection

Idea: Never Lose the Best — Re-diversify When Stuck

- **Elitism:** Preserve top 10 % of population each generation
- **Stagnation detection:** If best fitness is unchanged for 5 consecutive generations, inject fresh random individuals to re-diversify



Without Elitism (Standard GA)

A good solution in generation g can be completely lost if all offspring are worse — especially problematic with small time budgets.

Hybrid GA Mod 3: Threat-Proportional Mutation

Idea: Domain-Aware Mutation (Novel, WTA-Specific)

Instead of mutating to a **uniformly random** target, bias mutation toward targets with **high remaining threat**:

$$\text{remaining_threat}[j] = V_j \times \prod_{i:\, x_i = j} (1 - p_{ij})$$

Mutation target selection: $P(\text{mutate to } j) \propto \text{remaining_threat}[j]$

Standard Mutation

- Pick random target uniformly
- May reassign to an already well-covered target
- No domain knowledge used

Threat-Proportional Mutation

- High-value, poorly-covered targets attract more mutations
- Focuses search where improvement is **most needed**
- **Novel:** Our original WTA-specific contribution

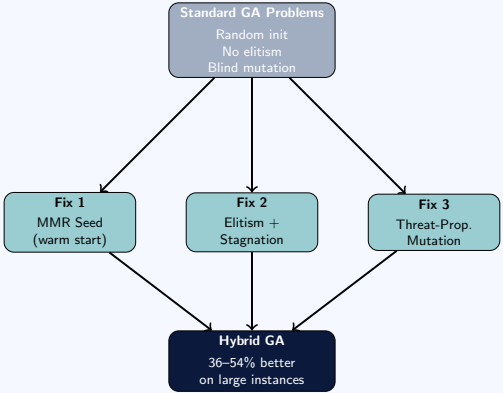
Concrete Example

If T1 ($V = 100$, survival$= 0.6$) and T2 ($V = 20$, survival$= 0.1$):

$\text{threat}_1 = 100 \times 0.6 = 60, \qquad \text{threat}_2 = 20 \times 0.1 = 2$

$\Rightarrow$ T1 is $30\times$ more likely to receive a mutated weapon than T2

Hybrid GA: Modifications Summary



Key Insight

Each modification targets a **specific weakness**. Together, they transform a struggling standard GA into a competitive solver that matches or beats MMR on all problem sizes.

Section 5

Experimental Setup

Dataset Design, Benchmark Protocol, and Setup

180 instances, 4 algorithms, 9 categories

Dataset Design: Custom Synthetic Benchmark

Why Design Our Own Dataset?

We created a synthetic dataset to systematically test **scaling behavior** across two dimensions: **problem size** (small/medium/large) and **resource availability** (balanced/scarce/rich).

Tier	Scenario	Weapons (W)	Targets (T)	Instances
Small	Balanced ($W = T$)	20	20	20
Small	Scarce ($W < T$)	10	20	20
Small	Rich ($W > T$)	30	20	20
Medium	Balanced	100	100	20
Medium	Scarce	50	100	20
Medium	Rich	150	100	20
Large	Balanced	250	250	20
Large	Scarce	125	250	20
Large	Rich	375	250	20

Total: 180 instances (9 categories $\times$ 20 instances each)

Dataset Parameters and Scenarios

Instance Parameters

- **Target values:** $V_j \sim U(10, 100)$ integers
- **Kill probabilities:** $p_{ij} \sim U(0.05, 0.90)$
- **RNG seed:** 42 (reproducible)
- **Format:** JSON files

Statistical Testing

- **Wilcoxon signed-rank test** ($\alpha = 0.05$) for pairwise algorithm comparison
- 20 instances per category ensures statistical validity

Scenario Rationale

Balanced ($W = T$): Standard benchmark; each weapon covers one target on average.

Scarce ($W < T$): Hardest scenario; not enough weapons for all targets. Unengaged targets have 100% survival.

Rich ($W > T$): Easiest scenario; surplus weapons allow thorough coverage. Lower objective values expected.

Algorithms Compared

Four algorithms evaluated on the same instances and time budgets

Algorithm	Description	Type
MMR (orig)	Maximum Marginal Return greedy heuristic. Faithful implementation from Hasan & Barua.	Heuristic
MMR-IR (mod)	MMR + tie-breaking + 1-opt local search + 2-opt swap search.	Enhanced Heuristic
GA (orig)	Standard Genetic Algorithm. Faithful implementation from Hasan & Barua.	Metaheuristic
Hybrid GA (mod)	GA + MMR seed + elitism + stagnation detection + threat-proportional mutation.	Enhanced Meta.

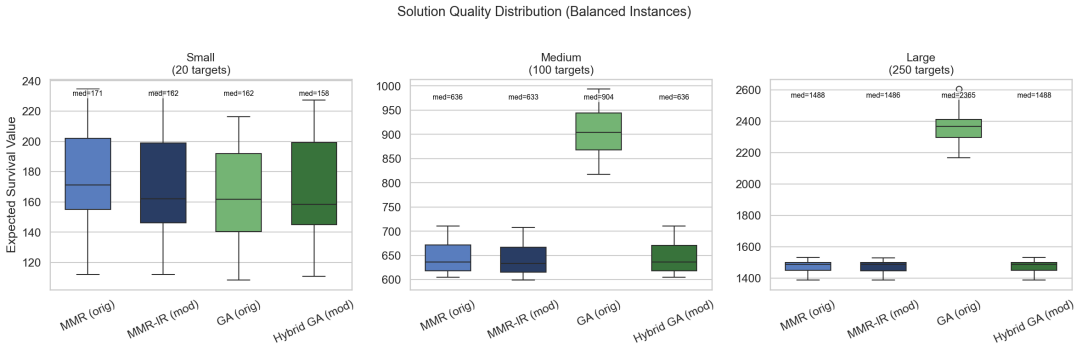
Results measured as expected survival value Z (lower = better)

Section 6

Experimental Results

Solution quality, scalability, and key findings

Solution Quality Distribution (Balanced)

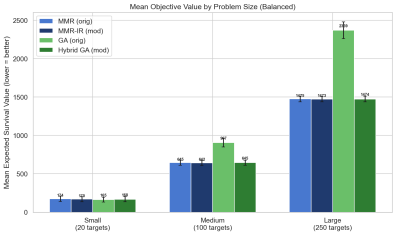


Distribution of objective values by algorithm and tier

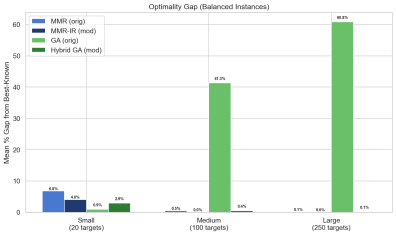
Result Description

- Dispersion increases with problem size, confirming large-tier instances are significantly harder.
- MMR-IR median is consistently below MMR, showing stable quality gain after local refinement.
- GA_Original shows high spread on medium/large tiers; Hybrid GA is both lower and tighter.

Mean Objective Value & Optimality Gap



Mean objective value (Z) by tier

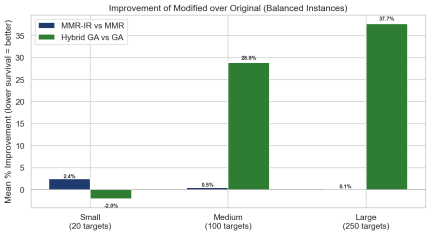


Gap from best-known solution

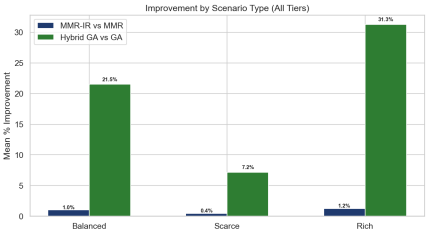
Result Description

- Objective quality gap between GA_Original and Hybrid GA widens as tier size grows.
- GA_Original has the largest gap from best-known, especially in medium/large tiers.
- Modified variants reduce both mean objective and gap, tracking near best-known performance.

Modified vs Original Gain: Tier & Scenario



Modified vs. original (% gain)

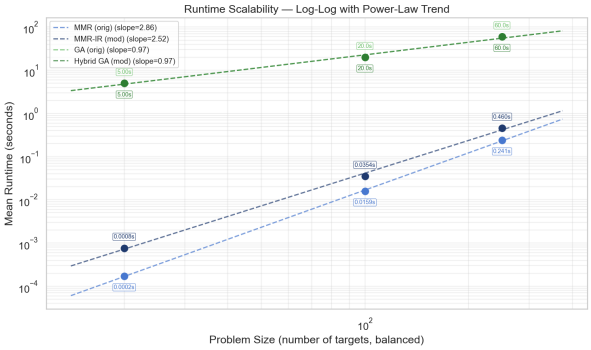


Mean improvement by scenario type

Result Description

- MMR-IR gives modest but consistent gains; Hybrid GA gives large gains on medium/large tiers.
- Improvement remains positive across balanced, scarce, and rich scenarios.
- This supports robustness of both modifications beyond a single operating condition.

Mean Runtime Scalability (Log-Log)

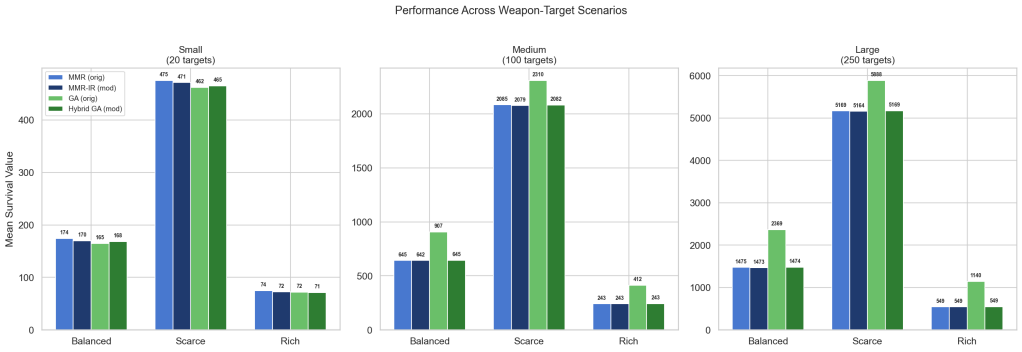


Mean runtime (seconds, log-log scale)

Result Description

- MMR/MMR-IR remain much faster across all sizes than GA-based methods.
- GA variants trend close to the time budget as size grows, while Hybrid GA still improves quality.
- Log-log slopes show scalable growth behavior and clear separation by algorithm family.

Scenario Sensitivity Across Tiers

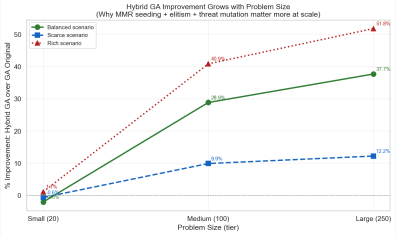


Balanced vs. scarce vs. rich, separated by tier

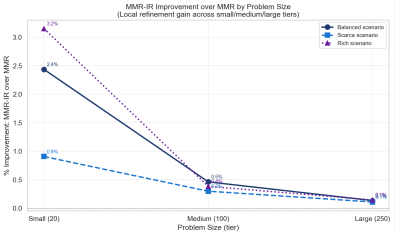
Result Description

- Scarce is consistently hardest (highest residual survival), rich is easiest, balanced is intermediate.
- Ranking is stable across tiers, confirming dataset design is behaviorally consistent.
- Hybrid GA keeps stronger quality than GA_Original in all three scenario types.

Scaling Gain Across Problem Size



Hybrid GA gain over GA_Original across small/medium/large



MMR-IR gain over MMR across small/medium/large

Result Description

- MMR-IR stays positive across all size tiers; Hybrid GA is mixed on small but strongly positive on medium/large.
- Hybrid GA exhibits the stronger scale effect, while MMR-IR provides consistent incremental gain.
- Showing both together highlights complementary impact: large GA uplift plus stable MMR refinement.

Why GA_Original Underperforms at Scale

The Scaling Problem

- **Population too large:** $pop_size = \max(W, T) = 375$ for large_rich
- **Only ≈160 generations** from a purely random start in a 60s budget
- **No elitism:** best solution can be overwritten between generations
- **Blind mutation:** uniform random target ignores high-threat areas

Our Fix (Hybrid GA)

- ✓ MMR-seeded initialisation gives a strong starting individual
- ✓ Elitism preserves top 10% each generation
- ✓ Stagnation detection injects diversity after 5 flat generations
- ✓ Threat-proportional mutation focuses on high-value under-covered targets

Key Findings

Summary of results across 720 experiment rows (180 instances $\times$ 4 algorithms)

MMR vs. MMR-IR

- Balanced-tier gains: **2.44%, 0.46%, 0.13%** (small/medium/large)
- Positive in all 9 categories
- Significant in **9/9** categories (Wilcoxon, $\alpha = 0.05$)

GA_Original vs. Hybrid GA

- Balanced-tier change: **-2.04%, 28.85%, 37.70%** (small/medium/large)
- Our 3 modifications directly address random init, elitism loss, and blind mutation
- Significant in **6/9** categories; all three small-tier categories are non-significant

Statistical validation: Wilcoxon signed-rank test ($\alpha = 0.05$): MMR-IR significant in 9/9 categories, Hybrid GA in 6/9. 20 instances per category.

Reproducibility: GitHub Repository

Single Master Script

All experiments are reproducible with a single command:

```
python run_all.py
```

Generates dataset → runs all 4 algorithms on 180 instances → produces 10 plots.

Repository Structure

```
CSE_462-Algorithm-Engineering-Sessional/
  codes/
    run_all.py            # Master script
    dataset_generator.py  # 180 instances
    experiment_runner.py  # Run all algorithms
    analysis.py           # 10 plots + stats
    mmr_original.py       # MMR (Hasan & Barua)
    mmr_modified.py       # MMR-IR (ours)
    ga_original.py        # GA (Hasan & Barua)
    ga_modified.py        # Hybrid GA (ours)
  datasets/               # JSON instances
  results/figures/        # 10 PNG plots
```

GitHub Link

GitHub Repository:

https://github.com/Arimotokika/CSE_462-Algorithm-Engineering-Sessional

Quick Reproduction Command

```
python run_all.py
```

Contains all code, datasets, and instructions required to reproduce the plots shown in this presentation.

References

[1] **Ahuja, R. K., et al. (2007).** Exact and heuristic algorithms for the weapon-target assignment problem. *Operations Research*.

[2] **Bertsimas, D., et al. (2025).** Solving large-scale WTA using branch-price-and-cut. *Naval Research Logistics*.

[3] **Andersen, A. C., et al. (2022).** WTA: Exact and approximate algorithms. *Annals of Operations Research*.

[4] **Camm, J. D., et al. (2002).** Nature reserve site selection. *Operations Research*.

[5] **Kwon, O., et al. (1999).** Lagrangian relaxation approach to targeting problem. *Naval Research Logistics*.

[6] **Kim, K., et al. (2022).** WTA model for ground combat using MINLP. *Mathematical Problems in Engineering*.

[7] **Kline, A. G., et al. (2020).** Heuristic and metaheuristic approaches for the static weapon-target assignment problem. *J. Global Optim*.

[8] **Sonuc, E., et al. (2020).** Weapon target assignment problem. *IntechOpen*.

[9] **Xin, B., et al. (2018).** Marginal-return-based heuristic for sensor-weapon-target assignment. *IEEE Trans. SMC*.

[10] **Lee, Z. J., et al. (2003).** Hybrid ACO-GA for WTA. *IDEAL 2003*.

[11] **Bogdanowicz, Z. R., et al. (2013).** Optimization of weapon-target pairings. *IEEE Trans. Cybernetics*.

[12] **Lee, Z. J., et al. (2002).** Immunity-based ACO for WTA. *Applied Soft Computing*.

[13] **Hu, X., et al. (2018).** Improved ACO algorithm for WTA. *Mathematical Problems in Engineering*.

[14] **Li, Y., et al. (2017).** Modified Pareto ACO for bi-objective WTA. *Int. J. Aerospace Engineering*.

[15] **Lloyd, S. P., et al. (1986).** Weapons allocation: a problem of many local optima. *IEEE*.

Thank You

Questions & Discussion

CSE 462 — Algorithm Engineering Sessional

Department of CSE, BUET